

ENTREGA RUTAS



UNIVERSIDAD COMPLUTENSE DE MADRID

FACULTAD DE CIENCIAS MATEMÁTICAS
MÁSTER EN INGENIERÍA MATEMÁTICA

PROFESOR: José María Ferrer Caja

ALUMNA: Siria Catherine Íñiguez Brito

ASIGNATURA: Modelos Determinísticos en Logística

CURSO: 2025-2026

Madrid, 1 de Diciembre de 2025

Contents

1	INTRODUCCIÓN	2
2	ENUNCIADO DEL PROBLEMA	3
3	PLANTEAMIENTO	4
3.1	Datos	4
3.2	Variables	5
3.3	Función objetivo	5
3.4	Restricciones	5
3.5	Imposibilidad de subciclos	8
4	EJEMPLO EN PYOMO	10
4.1	Enunciado	10
4.2	Código en python	11
4.3	Resultados Detallados de la Optimización	16
4.3.1	Rutas Óptimas Obtenidas	16
4.3.2	Tiempos y Cargas Relevantes de la Solución	17
4.3.3	Valor Óptimo	18

1 INTRODUCCIÓN

El presente documento desarrolla la formulación completa de un modelo de programación lineal entera (PLE) para un **Problema de Rutas de Vehículos (VRP)** con características específicas de tiempo y flujo de mercancía.

El modelo aborda la gestión de una flota de vehículos con **capacidad máxima** y **tiempo de disponibilidad inicial** limitados, los cuales deben atender a una red de clientes con demandas de **recogida** o **entrega**, partiendo y regresando todos a un único depósito.

Dada la naturaleza del problema, donde la mercancía de entrega debe cargarse en el depósito inicial, y no existe intercambio entre clientes, se establece un modelo de flujo de carga y tiempo. El objetivo principal es determinar las rutas óptimas que permitan **minimizar el instante de finalización máximo** de todas las rutas, buscando la mayor eficiencia temporal en la operación logística.

2 ENUNCIADO DEL PROBLEMA

Escribir un modelo de programación lineal entera para un problema de rutas con las siguientes características:

1. **Depósito Único:** Existe un único depósito desde el que los vehículos que se usen deben partir y al que deben regresar.
2. **Restricciones de Vehículos:**
 - Cada vehículo tiene una **capacidad máxima** definida.
 - Cada vehículo tiene un **instante en que empieza a estar disponible** (tiempo de inicio).
3. **Tipos de Clientes:** Algunos clientes requieren **entrega** y otros requieren **recogida**, pero ningún cliente requiere ambas cosas.
4. **Flujo de Mercancía:** No hay intercambio de mercancía entre clientes. Por lo tanto, cada vehículo debe partir del depósito con la mercancía que debe servir a sus clientes que requieran entrega.
5. **Duración de Trayectos:** Las duraciones de los trayectos no dependen de los vehículos (son constantes).
6. **Objetivo:** Se desea terminar el reparto lo antes posible, es decir, el objetivo es **minimizar el instante de finalización máximo de las rutas** de todos los vehículos.

3 PLANTEAMIENTO

3.1 Datos

Conjuntos

- $\mathcal{K} = \{1, \dots, V\}$: Conjunto de vehículos disponibles
- $\mathcal{N}_p$: Conjunto de clientes que requieren recogida (Pickup)
- $\mathcal{N}_d$: Conjunto de clientes que requieren entrega (Delivery)
- $\mathcal{N} = \mathcal{N}_p \cup \mathcal{N}_d$: Conjunto total de clientes
- $\mathcal{N}^+ = \mathcal{N} \cup \{D_{\text{ini}}, D_{\text{fin}}\}$: Conjunto de nodos (clientes y depósitos)

Definición de Depósitos

Los depósitos inicial y final son nodos ficticios que no representan clientes, sino los puntos obligatorios de inicio y fin de las rutas.

- $D_{\text{ini}} = 0$: Depósito inicial (nodo 0).
- $D_{\text{fin}} = n + m + 1$: Depósito final (nodo $n + m + 1$).

Se han definido D_{ini} y D_{fin} , en el contexto del problema representan al depósito único.

Parámetros

- q_k : capacidad máxima del vehículo $k \in \mathcal{K}$
- a_k : instante en que el vehículo $k \in \mathcal{K}$ está disponible
- t_{ij} : duración del trayecto de $i \in \mathcal{N}^+$ a $j \in \mathcal{N}^+$
- $p_i = \begin{cases} +d_i, & \text{si } i \in \mathcal{N}_p \text{ (recogida: aumenta la carga)} \\ -d_i, & \text{si } i \in \mathcal{N}_d \text{ (entrega: disminuye la carga)} \end{cases} : \forall i \in \mathcal{N}$

Constantes M (Valores Grandes)

Se requieren constantes M suficientemente grandes para linealizar las restricciones condicionales. Estos valores actúan como una cota superior segura para el dominio de las variables continuas.

- **Cotas para restricciones de tiempo:**
 - $M_{\text{time},k}^{\text{ini}}$: Utilizado para acoplar el tiempo de salida inicial $T_{D_{\text{ini}}}^k$.
 - $M_{\text{time1},k}^{i,j}$ y $M_{\text{time2},k}^{i,j}$: Utilizados en las dos restricciones de secuenciamiento para acoplar $T_j^k = T_i^k + t_{ij}$.
 - $M_{\text{time},k}^{\text{fin}}$: Utilizado para acoplar el tiempo de finalización $T_{D_{\text{fin}}}^k$.
- **Cotas para Restricciones de Capacidad:**
 - $M_{\text{cap1},k}^{i,j}$ y $M_{\text{cap2},k}^{i,j}$: Utilizados en las dos restricciones de actualización de carga para acoplar $C_j^k = C_i^k + p_j$.

3.2 Variables

- Z : Tiempo de finalización máximo de todas las rutas.
- T_i^k : Instante temporal en que el vehículo $k \in \mathcal{K}$ inicia la actividad en el nodo $i \in \mathcal{N}^+$. La interpretación de este instante varía según el nodo:
 - Si $i = D_{\text{ini}}$: Es el **tiempo de salida** del vehículo k desde el depósito inicial. (Se requiere que $T_{D_{\text{ini}}}^k = a_k$ si el vehículo se usa, y $T_{D_{\text{ini}}}^k = 0$ en caso contrario).
 - Si $i \in \mathcal{N}$: Es el **tiempo de inicio del servicio** (entrega o recogida) en el cliente i .
 - Si $i = D_{\text{fin}}$: Es el **tiempo de llegada** al depósito final, marcando la finalización de la ruta.
- $X_{ijk} = \begin{cases} 1, & \text{si el vehículo } k \in \mathcal{K} \text{ visita } j \in \mathcal{N}^+ \text{ inmediatamente después de } i \in \mathcal{N}^+, \\ 0, & \text{en otro caso} \end{cases}$
con $i \neq j$.
- C_i^k : Carga del vehículo $k \in \mathcal{K}$ después de visitar el nodo $i \in \mathcal{N}^+$.

3.3 Función objetivo

El problema busca minimizar el instante de finalización máximo de todas las rutas de los vehículos. Inicialmente, se plantea como una función objetivo no lineal de tipo min max:

$$\min \left\{ \max_{k \in \mathcal{K}} T_{D_{\text{fin}}}^k \right\}$$

Para convertir el modelo a una programación lineal entera (PLE) estándar, se emplea una técnica de linealización introduciendo una **variable auxiliar** continua, Z . Esta variable representa el tiempo máximo de finalización de todas las rutas y se añade al conjunto de variables de decisión.

Al minimizar esta nueva variable (Z) y añadir una **restricción de acoplamiento** que obliga a Z a ser mayor o igual que el tiempo de finalización de cualquier ruta k , se garantiza que el valor óptimo de Z será, precisamente, el valor máximo de $T_{D_{\text{fin}}}^k$ para todos los vehículos.

El modelo PLE final se define por:

- **Función Objetivo:** minimizar Z

3.4 Restricciones

0. Restricción del objetivo (Acoplamiento de Z)

Esta restricción de acoplamiento asegura que Z capture el tiempo de finalización máximo de todas las rutas.

$$Z \geq T_{D_{\text{fin}}}^k \quad \forall k \in \mathcal{K}$$

1. Visita única

Cada cliente debe ser visitado exactamente una vez por un único vehículo.

$$\sum_{k \in \mathcal{K}} \sum_{j \in \mathcal{N}^+ \setminus \{i\}} X_{ijk} = 1 \quad \forall i \in \mathcal{N}$$

$$\sum_{k \in \mathcal{K}} \sum_{i \in \mathcal{N}^+ \setminus \{j\}} X_{ijk} = 1 \quad \forall j \in \mathcal{N}$$

2. Uso del vehículo (Inicio y Fin en el Depósito)

Un vehículo parte del depósito inicial (D_{ini}) a lo sumo una vez y llega al depósito final (D_{fin}) a lo sumo una vez.

$$\sum_{j \in \mathcal{N}} X_{D_{\text{ini}},j,k} \leq 1 \quad \forall k \in \mathcal{K}$$

$$\sum_{i \in \mathcal{N}} X_{i,D_{\text{final}},k} \leq 1 \quad \forall k \in \mathcal{K}$$

$$X_{i,D_{\text{ini}},k} = 0 \quad \forall i \in \mathcal{N}^+, \forall k \in \mathcal{K}$$

$$X_{D_{\text{ini}},D_{\text{final}},k} = 0 \quad \forall k \in \mathcal{K}$$

3. Continuidad de rutas (Flujo)

Si un vehículo llega a un cliente (i), debe salir de ese cliente.

$$\sum_{j \in \mathcal{N}^+} X_{ijk} = \sum_{j \in \mathcal{N}^+} X_{jik} \quad \forall k \in \mathcal{K}, \forall i \in \mathcal{N}, i \neq j,$$

4. Restricciones de tiempo

Tiempo de salida del depósito inicial Estas restricciones establecen el valor del tiempo de salida del depósito inicial $T_{D_{\text{ini}}}^k$ de manera condicional: si el vehículo k es utilizado (es decir, si sale hacia algún cliente), entonces su tiempo de salida debe ser mayor o igual que su disponibilidad a_k ; si no es utilizado, su tiempo debe quedar fijado a 0.

$$T_{D_{\text{ini}}}^k \geq a_k \left(\sum_{j \in \mathcal{N}} X_{D_{\text{ini}},j,k} \right) \quad \forall k \in \mathcal{K}$$

$$T_{D_{\text{ini}}}^k \leq M_{\text{time},k}^{\text{ini}} \left(\sum_{j \in \mathcal{N}} X_{D_{\text{ini}},j,k} \right) \quad \forall k \in \mathcal{K}$$

donde la constante $M_{\text{time},k}^{\text{ini}}$ se define posteriormente.

Restricciones de secuenciamiento Estas restricciones aseguran que, cuando el vehículo recorre el arco (i, j) con $X_{ijk} = 1$, el tiempo de llegada a j sea igual al tiempo de salida de i más el tiempo de viaje t_{ij} .

$$T_i^k + t_{ij} - T_j^k \leq (1 - X_{ijk}) M_{time1,k}^{i,j} \quad \forall i, j \in \mathcal{N}^+, i \neq j, \forall k \in \mathcal{K}$$

$$T_i^k + t_{ij} - T_j^k \geq -(1 - X_{ijk}) M_{time2,k}^{i,j} \quad \forall i, j \in \mathcal{N}^+, i \neq j, \forall k \in \mathcal{K}$$

Tiempo de finalización de ruta El tiempo de llegada al depósito final sólo puede ser positivo si el vehículo efectivamente llega a dicho depósito:

$$T_{D_{\text{fin}}}^k \leq \left(\sum_{i \in \mathcal{N}} X_{i,D_{\text{fin}},k} \right) M_{time,k}^{\text{fin}} \quad \forall k \in \mathcal{K}$$

Unificación de las constantes Big-M de tiempo Todas las restricciones temporales emplean una única constante unificada M_k^{time} por vehículo, válida tanto para:

- tiempo inicial del vehículo,
- restricciones de secuenciamiento,
- tiempo de llegada al depósito final.

Sea t_{max} el mayor tiempo de viaje entre cualquier par de nodos:

$$t_{\text{max}} = \max_{u,v \in \mathcal{N}^+} t_{uv}.$$

El recorrido máximo posible de un vehículo k contiene a lo sumo $|\mathcal{N}| + 1$ tramos (salida desde el depósito inicial, visita de todos los clientes y llegada al depósito final). Además, un vehículo no puede comenzar antes de su tiempo de disponibilidad a_k .

Por tanto, una cota ajustada para todas las restricciones temporales es:

$$M_k^{\text{time}} = a_k + (|\mathcal{N}| + 1) t_{\text{max}}$$

5. Restricciones de capacidad (Flujo de carga)

Estas restricciones modelan la evolución de la carga que transporta cada vehículo a lo largo de su ruta.

Carga inicial en el depósito La carga inicial del vehículo k en el depósito de salida D_{ini} debe corresponder a la suma de las demandas absolutas de entrega (d_i para $i \in \mathcal{N}_d$) de todos aquellos clientes que el vehículo efectivamente servirá.

$$C_{D_{\text{in}}}^k = \sum_{i \in \mathcal{N}_d} d_i \left(\sum_{j \in \mathcal{N}^+ \setminus \{i\}} X_{ijk} \right) \quad \forall k \in \mathcal{K}$$

Actualización de carga La carga en el vehículo se actualiza de acuerdo con la demanda del nodo visitado. Si el vehículo viaja del nodo i al nodo j ($X_{ijk} = 1$), entonces:

$$C_j^k = C_i^k + p_j.$$

Mediante Big-M esto se modeliza con:

$$C_i^k + p_j - C_j^k \leq (1 - X_{ijk}) M_{cap1,k}^{i,j} \quad \forall i, j \in \mathcal{N}^+, i \neq j, \forall k \in \mathcal{K}$$

$$C_i^k + p_j - C_j^k \geq -(1 - X_{ijk}) M_{cap2,k}^{i,j} \quad \forall i, j \in \mathcal{N}^+, i \neq j, \forall k \in \mathcal{K}$$

Límites de capacidad La carga debe permanecer siempre dentro de los límites del vehículo. Si el vehículo no visita el nodo i , entonces la carga se fuerza a cero automáticamente.

$$0 \leq C_i^k \leq q_k \left(\sum_{j \in \mathcal{N}^+ \setminus \{D_in\}} X_{ijk} \right) \quad \forall i \in \mathcal{N}^+, \forall k \in \mathcal{K}$$

Unificación de las constantes Big-M de capacidad Todas las restricciones de capacidad pueden una única constante por vehículo, M_k^{cap} .

Dado que la carga en cualquier nodo satisface:

$$0 \leq C_i^k \leq q_k,$$

y que el cambio máximo de carga provocado por visitar un nodo es:

$$|p_j| \leq \max_{i \in \mathcal{N}} d_i,$$

una cota segura para la mayor variación posible entre dos nodos consecutivos es:

$$|C_i^k + p_j - C_j^k| \leq q_k + \max_{i \in \mathcal{N}} d_i.$$

Por tanto, definimos:

$$M_k^{\text{cap}} = q_k + \max_{i \in \mathcal{N}} d_i$$

6. Dominio de Variables

$$\begin{aligned} X_{ijk} &\in \{0, 1\} & \forall i, j \in \mathcal{N}^+, i \neq j, \forall k \in \mathcal{K} \\ C_i^k &\geq 0 & \forall i \in \mathcal{N}^+, \forall k \in \mathcal{K} \\ T_i^k &\geq 0 & \forall i \in \mathcal{N}^+, \forall k \in \mathcal{K} \\ Z &\geq 0 \end{aligned}$$

3.5 Imposibilidad de subciclos

La eliminación de subciclos (circuitos cerrados entre clientes que no incluyen los depósitos) se debe a la **monotonía estricta** impuesta por las restricciones de secuenciamiento de tiempo.

1. Monotonía del Tiempo ($T_j > T_i$)

El modelo impone la siguiente relación cuando el arco $i \rightarrow j$ es utilizado ($X_{ijk} = 1$):

$$T_j = T_i + t_{ij}$$

Donde t_{ij} es el tiempo de viaje. Dado que los tiempos de viaje (t_{ij}) deben ser **positivos** ($t_{ij} > 0$), esta igualdad obliga a que el tiempo siempre **aumente estrictamente** a lo largo de cualquier ruta:

$$T_j > T_i$$

El vehículo siempre debe llegar a un nodo posterior en un instante de tiempo mayor. El tiempo es, por definición, creciente a lo largo de la ruta.

2. La Contradicción Lógica del Subciclo

Si se intentara crear un subciclo de N nodos (clientes) que no pase por los depósitos, representamos este ciclo como una secuencia: $i_1 \rightarrow i_2 \rightarrow \dots \rightarrow i_N \rightarrow i_1$, las restricciones de secuenciamiento, al imponer la condición de monotonía estricta ($T_j > T_i$) en cada arco del ciclo, requerirían la siguiente cadena de desigualdades temporales:

- $i_1 \rightarrow i_2$ requiere: $T_{i_2} > T_{i_1}$
- $i_2 \rightarrow i_3$ requiere: $T_{i_3} > T_{i_2}$
- ...
- $i_N \rightarrow i_1$ requiere: $T_{i_1} > T_{i_N}$

Al encadenar todas las N desigualdades (sumando las duraciones t_{ij} en cada paso), se obtiene la relación final:

$$T_{i_1} < T_{i_2} < T_{i_3} < \dots < T_{i_N} < T_{i_1}$$

Esta cadena conduce a la **contradicción insalvable** $T_{i_1} > T_{i_1}$. De esta manera, las variables de tiempo no solo miden el flujo temporal, sino que también actúan como un mecanismo de ordenación que impide cualquier circuito cerrado. La eliminación de subciclos es válida porque todos los tiempos entre nodos distintos cumplen $t_{ij} > 0$.

4 EJEMPLO EN PYOMO

Esta sección presenta la resolución de un problema de rutas de vehículos como se describe en la sección 3 con características específicas de entrega y recogida, utilizando Pyomo como herramienta de modelado y optimización.

4.1 Enunciado

Se requiere planificar la logística de una flota de vehículos para atender a cinco clientes (identificados como nodos del 2 al 6) desde un único almacén central (nodo 1). El objetivo principal de la operación es minimizar el tiempo total de finalización de la operación, determinando a su vez la secuencia óptima de visitas para cada vehículo de la flota, asegurando que se satisfagan todas las demandas dentro de las restricciones de capacidad y disponibilidad.

Las demandas (en hectolitros) son:

Cliente	2	3	4	5	6
Entrega	0	3	4	0	2
Recogida	4	0	0	2	0

Table 1: Demandas de Clientes (en hectolitros)

Los tiempos (expresados en minutos) por viajar de un nodo a otro son:

Tiempo	1	2	3	4	5	6
1	0	90	120	120	180	180
2	-	0	30	60	150	120
3	-	-	0	30	150	150
4	-	-	-	0	90	120
5	-	-	-	-	0	60
6	-	-	-	-	-	0

Table 2: Tiempos de Viaje t_{ij} (en minutos)

Nota: Se asume que los tiempos son cero si el nodo es el mismo. En otro caso, para viajes de j a i donde t_{ij} no está dado, se asumirá que $t_{ji} = t_{ij}$.

Se dispone de tres vehículos cuyas capacidades (en hectolitros) e instantes en que empiezan a estar disponibles son:

Vehículo	1	2	3
Capacidad (hl)	6	5	4
Instante Disp. (min)	60	30	0

Table 3: Datos de Vehículos

Nota: En el ejemplo numérico los clientes están indexados como 2–6, pero en la implementación Pyomo se renumeran como 1–5.

4.2 Código en python

La implementación del modelo de programación lineal entera se ha desarrollado en python, en un archivo llamado PracticaRutas.py.

```
1
2 # MODELO DE RUTAS DE VEHICULOS CON CAPACIDAD Y TIEMPOS DE DISPONIBILIDAD
3 #
4 # Problema: Planificar las rutas para 3 vehiculos (K) para atender a 5
   clientes (Nodos 1 a 5) desde un deposito (D_ini/D_fin), con el
   objetivo de MINIMIZAR Z, tiempo total de finalizacion de la ultima
   ruta).
5
6 # Caracteristicas de la Instancia:
7 # Flota: 3 vehiculos con distintas capacidades y diferentes tiempos de
   inicio de operacion.
8 # Demandas: Mixtas (Entrega y Recogida).
9 # - Np (/Recogida, p > 0): Clientes {1, 4}.
10 # - Nd (Entrega, p < 0): Clientes {2, 3, 5}.
11 #
12 # Variables:
13 # - X[i,j,k]: 1 si el vehiculo k viaja del nodo i al j.
14 # - T[i,k]: Tiempo de llegada del vehiculo k al nodo i.
15 # - C[i,k]: Carga del vehiculo k al salir del nodo i.
16 # - Z: Tiempo objetivo a minimizar.
17
18 import pyomo.environ as pyo
19 from pyomo.opt import SolverFactory
20
21 # -----
22 # 1. DATOS
23 # -----
24
25 K = ['V1', 'V2', 'V3']
26 Np = [1,4]
27 Nd = [2,3,5]
28 N = sorted(set(Np) | set(Nd))
29 Nmas = N + ['D_ini', 'D_fin']
30
31 q = {'V1':6, 'V2':5, 'V3':4}
32 a = {'V1':60, 'V2':30, 'V3':0}
33 dem = {1:4, 2:3, 3:4, 4:2, 5:2}
34 p = {i: (dem[i] if i in Np else -dem[i]) for i in N} # p_i con signo
35
36 t = {
37     (1,1):0, (1,2):30, (1,3):60, (1,4):150, (1,5):120,
38     (2,1):30, (2,2):0, (2,3):30, (2,4):150, (2,5):150,
39     (3,1):60, (3,2):30, (3,3):0, (3,4):90, (3,5):120,
40     (4,1):150, (4,2):150, (4,3):90, (4,4):0, (4,5):60,
41     (5,1):120, (5,2):150, (5,3):120, (5,4):60, (5,5):0,
42     ('D_ini',1):90, ('D_ini',2):120, ('D_ini',3):120, ('D_ini',4):180, ('
   'D_ini',5):180,
43     (1,'D_fin'):90, (2,'D_fin'):120, (3,'D_fin'):120, (4,'D_fin'):180,
   (5,'D_fin'):180,
44     ('D_ini','D_fin'):0, ('D_fin','D_ini'):0, ('D_ini','D_ini'):0, ('
   D_fin','D_fin'):0,
45 }
```

```

46
47 for i in N:
48     t[(i, 'D_ini')] = t[( 'D_ini', i)]
49     t[( 'D_fin', i)] = t[(i, 'D_fin')]
50
51
52 # -----
53 # 2. Big-M AJUSTADAS
54 # -----
55
56 max_t = max(t.values())
57 num_clients = len(N)
58
59 #Big M para el tiempo
60 M_time = {k: a[k] + (num_clients + 1) * max_t for k in K}
61
62 #Big M para la capacidad
63 M_cap = {k: q[k] + max(dem.values()) for k in K}
64
65
66 # -----
67 # 3. MODELO
68 # -----
69
70 model = pyo.ConcreteModel()
71
72 model.K = pyo.Set(initialize=K)
73 model.N = pyo.Set(initialize=N)
74 model.Nmas = pyo.Set(initialize=Nmas)
75 model.Np = pyo.Set(initialize=Np)
76 model.Nd = pyo.Set(initialize=Nd)
77
78 model.q = pyo.Param(model.K, initialize=q)
79 model.a = pyo.Param(model.K, initialize=a)
80 model.dem = pyo.Param(model.N, initialize=dem)
81
82 # Param p en Nmas
83 p_ext = {i: (dem[i] if i in Np else -dem[i]) for i in N}
84 p_ext['D_ini'] = 0
85 p_ext['D_fin'] = 0
86 model.p = pyo.Param(model.Nmas, initialize=p_ext)
87
88 # Param t en Nmas x Nmas
89 model.t = pyo.Param(model.Nmas, model.Nmas, initialize=t)
90
91 # Ms como Params
92 model.M_time = pyo.Param(model.K, initialize=M_time)
93 model.M_cap = pyo.Param(model.K, initialize=M_cap)
94
95 # Variables
96 model.X = pyo.Var(model.Nmas, model.Nmas, model.K, domain=pyo.Binary)
97 model.T = pyo.Var(model.Nmas, model.K, domain=pyo.NonNegativeReals)
98 model.C = pyo.Var(model.Nmas, model.K, domain=pyo.NonNegativeReals)
99 model.Z = pyo.Var(domain=pyo.NonNegativeReals)
100
101 # Objetivo
102 model.obj = pyo.Objective(expr=model.Z, sense=pyo.minimize)
103

```

```

104
105 # -----
106 # 4. RESTRICCIONES
107 # -----
108
109 # ---- (0) ACOPLAMIENTO Z >= T_Dfin
110 def Res0(model,k):
111     return model.Z >= model.T['D_fin',k]
112 model.Res0 = pyo.Constraint(model.K, rule=Res0)
113
114
115 # ---- (1) VISITA UNICA
116 def Res11(model,i):
117     return sum(model.X[i,j,k] for j in model.Nmas if j!=i for k in model
118         .K) == 1
119 model.Res11 = pyo.Constraint(model.N, rule=Res11)
120
121 def Res12(model,j):
122     return sum(model.X[i,j,k] for i in model.Nmas if i!=j for k in model
123         .K) == 1
124 model.Res12 = pyo.Constraint(model.N, rule=Res12)
125
126 # ---- (2) DEPOSITOS
127 def Res21(model,k):
128     return sum(model.X['D_ini',j,k] for j in model.N) <= 1
129 model.Res21 = pyo.Constraint(model.K, rule=Res21)
130
131 def Res22(model,k):
132     return sum(model.X[i,'D_fin',k] for i in model.N) <= 1
133 model.Res22 = pyo.Constraint(model.K, rule=Res22)
134
135 def Res23(model,i,k):
136     # prohibir entrar al deposito inicial
137     if i != 'D_ini':
138         return model.X[i,'D_ini',k] == 0
139     return pyo.Constraint.Skip
140 model.Res23 = pyo.Constraint(model.Nmas, model.K, rule=Res23)
141
142 def Res24(model,k):
143     return model.X['D_ini','D_fin',k] == 0
144 model.Res24 = pyo.Constraint(model.K, rule=Res24)
145
146 # ---- (3) FLUJO CONTINUO
147 def Res3(model,i,k):
148     return sum(model.X[i,j,k] for j in model.Nmas if j!=i) == sum(model.
149         X[j,i,k] for j in model.Nmas if j!=i)
150 model.Res3 = pyo.Constraint(model.N, model.K, rule=Res3)
151
152 # ---- (4) TIEMPOS
153 def Res41(model,k):
154     return model.T['D_ini',k] >= model.a[k] * sum(model.X['D_ini',j,k]
155         for j in model.N)
156 model.Res41 = pyo.Constraint(model.K, rule=Res41)
157

```

```

158
159 def Res42(model,k):
160     return model.T['D_ini',k] <= model.M_time[k] * sum(model.X['D_ini',j
,k] for j in model.N)
161 model.Res42 = pyo.Constraint(model.K, rule=Res42)
162
163 def Res_seq1(model,i,j,k):
164     if i != j:
165         return model.T[i,k] + model.t[i,j] - model.T[j,k] <= (1-model.X[
i,j,k]) * model.M_time[k]
166     return pyo.Constraint.Skip
167 model.Res_seq1 = pyo.Constraint(model.Nmas, model.Nmas, model.K, rule=
Res_seq1)
168
169 def Res_seq2(model,i,j,k):
170     if i != j:
171         return model.T[i,k] + model.t[i,j] - model.T[j,k] >= -(1-model.X
[i,j,k]) * model.M_time[k]
172     return pyo.Constraint.Skip
173 model.Res_seq2 = pyo.Constraint(model.Nmas, model.Nmas, model.K, rule=
Res_seq2)
174
175 def Res45(model,k):
176     return model.T['D_fin',k] <= model.M_time[k] * sum(model.X[i,'D_fin'
,k] for i in model.N)
177 model.Res45 = pyo.Constraint(model.K, rule=Res45)
178
179
180 # ---- (5) CAPACIDAD
181 def Res51(model,k):
182     return model.C['D_ini',k] == sum(model.dem[i] * sum(model.X[i,j,k]
for j in model.Nmas if j!=i) for i in model.Nd)
183 model.Res51 = pyo.Constraint(model.K, rule=Res51)
184
185 def Res52(model,i,j,k):
186     if i != j:
187         return model.C[i,k] + model.p[j] - model.C[j,k] <= (1-model.X[i,
j,k]) * model.M_cap[k]
188     return pyo.Constraint.Skip
189 model.Res52 = pyo.Constraint(model.Nmas, model.Nmas, model.K, rule=Res52
)
190
191 def Res53(model,i,j,k):
192     if i != j:
193         return model.C[i,k] + model.p[j] - model.C[j,k] >= -(1-model.X[i
,j,k]) * model.M_cap[k]
194     return pyo.Constraint.Skip
195 model.Res53 = pyo.Constraint(model.Nmas, model.Nmas, model.K, rule=Res53
)
196
197 def Res54(model,i,k):
198     return model.C[i,k] <= model.q[k] * sum(model.X[i,j,k] for j in
model.Nmas if j != 'D_ini')
199 model.Res54 = pyo.Constraint(model.Nmas, model.K, rule=Res54)
200
201
202
203

```

```

204
205 # =====
206 # 6. SOLVER
207 # =====
208
209 solver = SolverFactory('glpk')
210
211 res = solver.solve(model, tee=True)
212
213 print("\n===== DETALLE DE VARIABLES =====")
214 model.display()
215
216
217 # =====
218 # 7. IMPRESION DE LAS RUTAS
219 # =====
220
221 def Ruta (model, k):
222     """Reconstruye la ruta secuencialmente a partir de X."""
223     start = None
224     for j in model.N:
225         if model.X['D_ini',j,k].value == 1:
226             start = j
227             break
228     if start is None:
229         return ["Vehiculo no usado"]
230
231     ruta = ['D_ini', start]
232     actual = start
233
234     while actual != 'D_fin':
235         next_node = None
236         for j in model.Nmas:
237             if j != actual and model.X[actual,j,k].value == 1:
238                 next_node = j
239                 ruta.append(j)
240                 actual = j
241                 break
242         if next_node is None:
243             break
244
245     return ruta
246
247
248 print("\n===== RUTAS OBTENIDAS =====")
249 for k in K:
250     ruta = Ruta(model, k)
251     if ruta == ["Vehiculo no usado"]:
252         print(f"\nVehiculo {k}: NO USADO")
253     else:
254         pretty = " ".join(str(x) for x in ruta)
255         print(f"\nVehiculo {k}: {pretty}")
256
257 print("\nValor optimo Z:", model.Z.value)
258
259
260
261

```



```

262
263 # =====
264 # 8. DETALLE DE TIEMPOS Y CARGAS
265 # =====
266
267 print("\n===== TIEMPOS Y CARGAS CLAVE =====")
268 for k in K:
269     # Comprobar si el vehiculo fue utilizado
270     ruta = Ruta(model, k)
271     if ruta != ["Vehiculo no usado"]:
272         # T_ini,k : Tiempo de salida del deposito inicial (D_ini)
273         t_ini_k = model.T['D_ini', k].value
274
275         # T_fin,k : Tiempo de llegada al deposito final (D_fin)
276         t_fin_k = model.T['D_fin', k].value
277
278         # C_ini,k : Carga al salir del deposito inicial (D_ini)
279         c_ini_k = model.C['D_ini', k].value
280
281         print(f"\n--- Vehiculo {k} (Capacidad: {model.q[k]},
282             Disponibilidad: {model.a[k]}) ---")
283         print(f"Ruta: {' '.join(str(x) for x in ruta)}")
284         print(f"Carga Inicial (C_ini): {c_ini_k}")
285         print(f"Tiempo de Inicio (T_ini): {t_ini_k}")
286         print(f"Tiempo de Finalizacion (T_fin): {t_fin_k}")
287     else:
288         print(f"\n--- Vehiculo {k}: NO USADO ---")
289
290 print("\nValor optimo Z (Max de T_fin):", model.Z.value)

```

Listing 1: Script Pyomo para el VRP con Tiempos y Capacidad

4.3 Resultados Detallados de la Optimización

El modelo implementado en pyomo ha sido resuelto mediante el solver **glpk**. Los resultados arrojados por el solver mantienen la notación de variables definida en la Sección 3 (Planteamiento), y la solución final se resume a continuación.

```

===== RUTAS OBTENIDAS =====
Vehículo V1: D_ini → 1 → 2 → D_fin
Vehículo V2: D_ini → 5 → D_fin
Vehículo V3: D_ini → 3 → 4 → D_fin
Valor óptimo Z: 390.0

```

Figure 1: Salida del solver Pyomo para las rutas.

4.3.1 Rutas Óptimas Obtenidas

La solución óptima del modelo de programación entera define las siguientes rutas para los vehículos, reconstruidas con la notación específica del problema (nodo inicial 1 y clientes del 2 al 6).

Rutas Óptimas Obtenidas	
Vehículo	Ruta
V1	1 → 2 → 3 → 1
V2	1 → 6 → 1
V3	1 → 4 → 5 → 1

4.3.2 Tiempos y Cargas Relevantes de la Solución

La Figura 2 muestra la salida del solver con los valores para las cargas iniciales (C_{ini}^k) y los tiempos de inicio (T_{ini}^k) y finalización (T_{fin}^k). Estos valores son fundamentales para determinar el tiempo total de la operación.

```

===== TIEMPOS Y CARGAS CLAVE =====

--- Vehículo V1 (Capacidad: 6, Disponibilidad: 60) ---
Ruta: D_ini → 1 → 2 → D_fin
Carga Inicial (C_ini): 3.0
Tiempo de Inicio (T_ini): 60.0
Tiempo de Finalización (T_fin): 300.0

--- Vehículo V2 (Capacidad: 5, Disponibilidad: 30) ---
Ruta: D_ini → 5 → D_fin
Carga Inicial (C_ini): 2.0
Tiempo de Inicio (T_ini): 30.0
Tiempo de Finalización (T_fin): 390.0

--- Vehículo V3 (Capacidad: 4, Disponibilidad: 0) ---
Ruta: D_ini → 3 → 4 → D_fin
Carga Inicial (C_ini): 4.0
Tiempo de Inicio (T_ini): 0.0
Tiempo de Finalización (T_fin): 390.0

Valor óptimo Z (Max de T_fin): 390.0

```

Figure 2: Salida del solver Pyomo para las cargas y tiempos.

A partir de los datos proporcionados por el solver, se resumen los parámetros logísticos más relevantes:

- **Vehículo V1 (Capacidad: 6, Disponibilidad: 60):**
 - Carga Inicial (C_{ini}^1): 3.0 unidades.
 - Tiempo de Inicio (T_{ini}^1): 60.0 minutos.
 - Tiempo de Finalización (T_{fin}^1): 300.0 minutos.
- **Vehículo V2 (Capacidad: 5, Disponibilidad: 30):**
 - Carga Inicial (C_{ini}^2): 2.0 unidades.
 - Tiempo de Inicio (T_{ini}^2): 30.0 minutos.
 - Tiempo de Finalización (T_{fin}^2): 390.0 minutos.

- **Vehículo V3 (Capacidad: 4, Disponibilidad: 0):**

- Carga Inicial (C_{ini}^3): 4.0 unidades.
- Tiempo de Inicio (T_{ini}^3): 0.0 minutos.
- Tiempo de Finalización (T_{fin}^3): 390.0 minutos.

4.3.3 Valor Óptimo

El valor óptimo (Z) del problema, que representa el tiempo mínimo requerido para completar todas las tareas es:

$$\text{Valor óptimo } Z = \max(300.0, 390.0, 390.0) = 390.0 \text{ minutos} \quad (1)$$

Este valor de **390.0 minutos** es el tiempo en el que el último vehículo retorna al depósito, marcando la finalización de toda la operación de entrega y recogida.